

AI Assisted Coding Assignment- 11.2

RUDROJU RUPA SRI

2303A51918

BATCH-30

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Lab Objectives

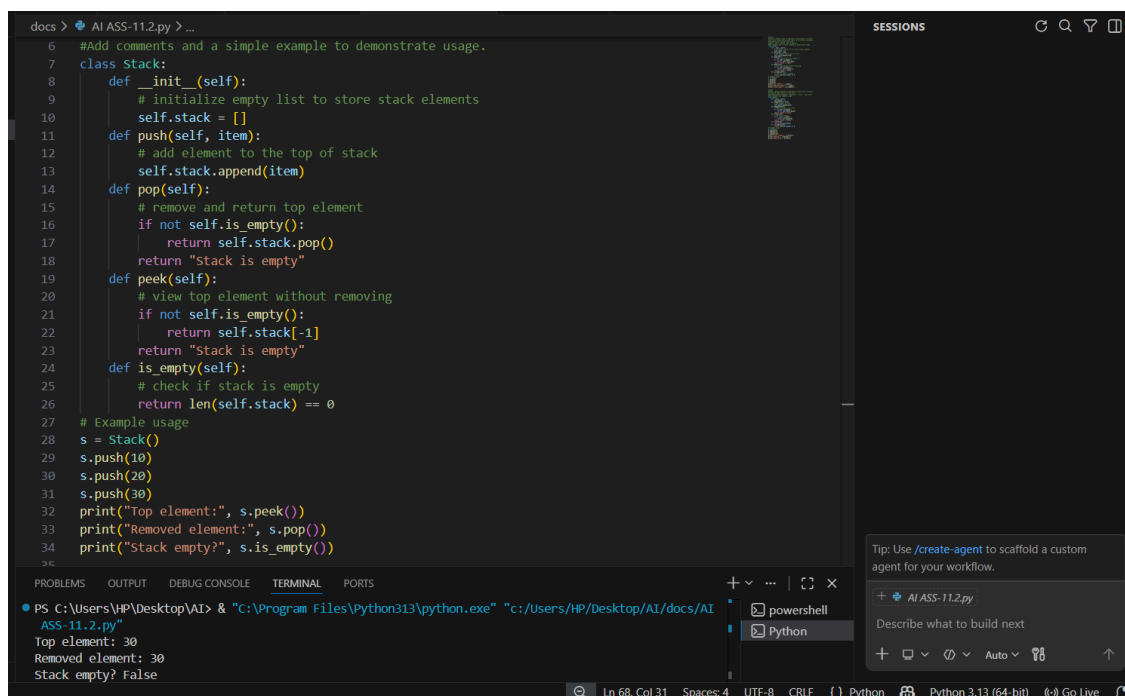
- Use AI to assist in designing and implementing fundamental data structures in Python.
- Learn how to prompt AI for structure creation, optimization, and documentation.
- Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.
- Enhance code quality with AI-generated comments and performance suggestions.

Task Description -1 – (Stack Using AI Guidance)

- Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Expected Output:

- A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.



The screenshot displays a code editor with a Python file named `AI ASS-11.2.py`. The code defines a `Stack` class with methods for `push`, `pop`, `peek`, and `is_empty`. It includes example usage code that pushes 10, 20, and 30 onto the stack, then prints the top element, removes it, and checks if the stack is empty. The terminal output shows the execution results: `Top element: 30`, `Removed element: 30`, and `Stack empty? False`. The editor interface includes a sidebar with file explorer, a search bar, and a session panel on the right.

```
docs > AI ASS-11.2.py > ...
6 #Add comments and a simple example to demonstrate usage.
7 class Stack:
8     def __init__(self):
9         # initialize empty list to store stack elements
10        self.stack = []
11    def push(self, item):
12        # add element to the top of stack
13        self.stack.append(item)
14    def pop(self):
15        # remove and return top element
16        if not self.is_empty():
17            return self.stack.pop()
18        return "Stack is empty"
19    def peek(self):
20        # view top element without removing
21        if not self.is_empty():
22            return self.stack[-1]
23        return "Stack is empty"
24    def is_empty(self):
25        # check if stack is empty
26        return len(self.stack) == 0
27
28 # Example usage
29 s = Stack()
30 s.push(10)
31 s.push(20)
32 s.push(30)
33 print("Top element:", s.peek())
34 print("Removed element:", s.pop())
35 print("Stack empty?", s.is_empty())
```

Terminal Output:

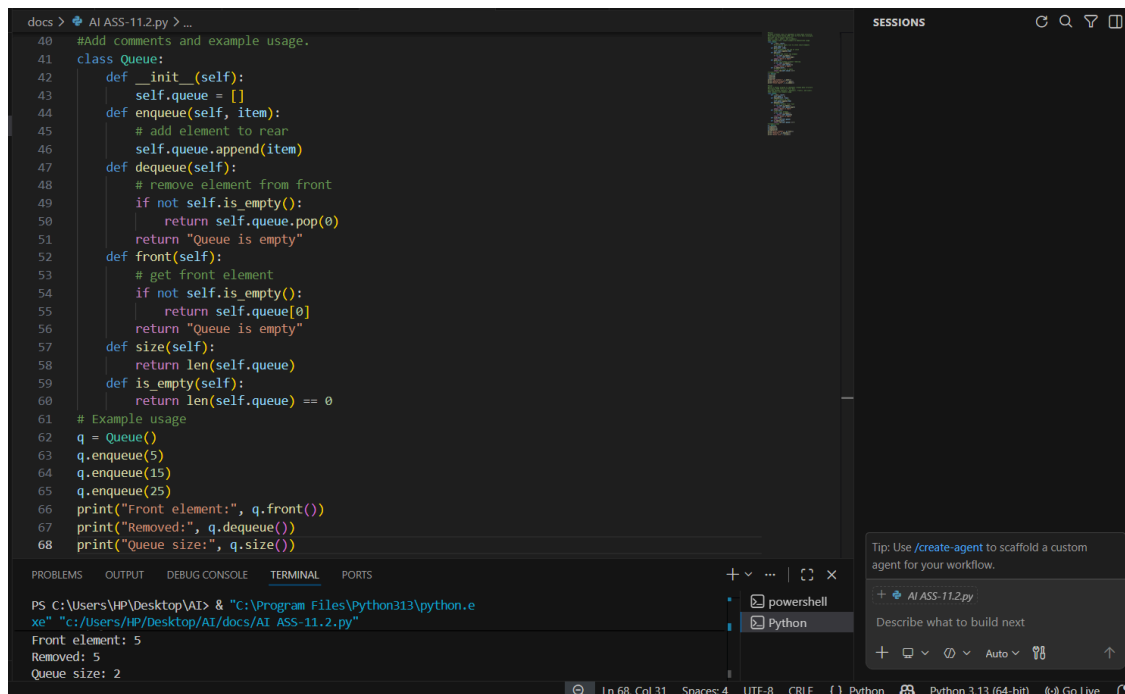
```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:\Users\HP\Desktop\AI\AI ASS-11.2.py"
Top element: 30
Removed element: 30
Stack empty? False
```

Task Description -2 – (Queue Design)

- Task: Use AI assistance to create a Queue data structure following FIFO principles

Expected Output:

- A complete Queue implementation including enqueue, dequeue, front element access, and size calculation



The screenshot displays a VS Code editor with a Python file named `AI ASS-11.2.py`. The code implements a `Queue` class using a list to store elements. The class methods include `__init__` (initializes an empty list), `enqueue` (appends an element to the rear), `dequeue` (removes and returns the front element, returning "Queue is empty" if the queue is empty), `front` (returns the front element or "Queue is empty" if the queue is empty), `size` (returns the length of the queue), and `is_empty` (returns True if the queue is empty). An example usage section demonstrates the queue's behavior by enqueuing 5, 15, and 25, then printing the front element (5), dequeuing it, and printing the new size (2).

```
docs > AI ASS-11.2.py > ...
40 #Add comments and example usage.
41 class Queue:
42     def __init__(self):
43         self.queue = []
44     def enqueue(self, item):
45         # add element to rear
46         self.queue.append(item)
47     def dequeue(self):
48         # remove element from front
49         if not self.is_empty():
50             return self.queue.pop(0)
51         return "Queue is empty"
52     def front(self):
53         # get front element
54         if not self.is_empty():
55             return self.queue[0]
56         return "Queue is empty"
57     def size(self):
58         return len(self.queue)
59     def is_empty(self):
60         return len(self.queue) == 0
61 # Example usage
62 q = Queue()
63 q.enqueue(5)
64 q.enqueue(15)
65 q.enqueue(25)
66 print("Front element:", q.front())
67 print("Removed:", q.dequeue())
68 print("Queue size:", q.size())
```

The terminal output shows the execution results:

```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:\Users\HP\Desktop\AI\docs\AI ASS-11.2.py"
Front element: 5
Removed: 5
Queue size: 2
```

The interface also includes a sidebar with a file explorer, a search bar, and a panel with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. A tip at the bottom right suggests using `/create-agent` to scaffold a custom agent.

Task Description -3 –(Singly Linked List Construction)

- Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

- Correctly functioning linked list with node creation, insertion logic, and display functionality.

```
docs > AI ASS-11.2.py > Node > _init_  
72 #include node creation, insertion at the end, and traversal  
73 #({display the list}). Add comments and example usage.  
74 class Node:  
75     def __init__(self, data):  
76         self.data = data  
77         self.next = None  
78 class LinkedList:  
79     def __init__(self):  
80         self.head = None  
81     def insert(self, data):  
82         new_node = Node(data)  
83         if self.head is None:  
84             self.head = new_node  
85             return  
86         temp = self.head  
87         while temp.next:  
88             temp = temp.next  
89         temp.next = new_node  
90     def display(self):  
91         temp = self.head  
92         while temp:  
93             print(temp.data, end=" -> ")  
94             temp = temp.next  
95         print("None")  
96 # Example usage  
97 ll = LinkedList()  
98 ll.insert(1)  
99 ll.insert(2)  
100 ll.insert(3)  
101 ll.display()
```

Tip: Use `/create-agent` to scaffold a custom agent for your workflow.

Describe what to build next

1 -> 2 -> 3 -> None

PS C:\Users\HP\Desktop\AI>

Task Description -4 – (Binary Search Tree Operations)

- Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Expected Output:

- BST program with correct node insertion and in-order traversal output.

The screenshot shows a VS Code editor with a file named `AI ASS-11.2.py` open. The code implements a Binary Search Tree (BST) with methods for insertion, in-order traversal, and example usage. The terminal output shows the execution of the script, resulting in an in-order traversal of the tree containing the values 20, 30, 40, 50, and 70.

```
docs > AI ASS-11.2.py > BSTNode > _init_
105 #include node insertion and in-order traversal.
106 #Add comments and example usage.
107 class BSTNode:
108     def __init__(self, key):
109         self.left = None
110         self.right = None
111         self.value = key
112     def insert(root, key):
113         if root is None:
114             return BSTNode(key)
115         if key < root.value:
116             root.left = insert(root.left, key)
117         else:
118             root.right = insert(root.right, key)
119         return root
120     def inorder(root):
121         if root:
122             inorder(root.left)
123             print(root.value, end=" ")
124             inorder(root.right)
125 # Example usage
126 root = None
127 root = insert(root, 50)
128 insert(root, 30)
129 insert(root, 20)
130 insert(root, 40)
131 insert(root, 70)
132 print("Inorder traversal:")
133 inorder(root)

PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:\Users\HP\Desktop\AI\docs\AI ASS-11.2.py"
Inorder traversal:
20 30 40 50 70
PS C:\Users\HP\Desktop\AI>
```

Task Description -5 – (Hash Table Implementation)

- Task: Create a hash table using AI with collision handling

Expected Output:

- Hash table supporting insert, search, and delete using chaining or open

The screenshot shows a VS Code editor with a file named `AI ASS-11.2.py` open. The code implements a HashTable using chaining for collision handling, with methods for insertion, search, and deletion. The terminal output shows the execution of the script, resulting in the following output: Key found, Key deleted, Key not found, and Key not found.

```
docs > AI ASS-11.2.py > HashTable > _init_
135 inorder(root)
136 print() # move to next line
137
138
139 #Task5
140 #Create a Python Hash Table Implementation using chaining for
141 #collision handling. Include insert, search, and delete methods.
142 #Add comments and demonstrate usage.
143 class HashTable:
144     def __init__(self, size):
145         self.size = size
146         self.table = [[] for _ in range(size)]
147     def hash_function(self, key):
148         return key % self.size
149     def insert(self, key):
150         index = self.hash_function(key)
151         self.table[index].append(key)
152     def search(self, key):
153         index = self.hash_function(key)
154         if key in self.table[index]:
155             return "Key found"
156         return "Key not found"
157     def delete(self, key):
158         index = self.hash_function(key)
159         if key in self.table[index]:
160             self.table[index].remove(key)
161         return "Key deleted"
162         return "Key not found"
163 # Example usage
164 ht = HashTable(10)
165 ht.insert(15)
166 ht.insert(25)
167 ht.insert(35)
168 print(ht.search(25))
169 print(ht.delete(25))
170 print(ht.search(25))

PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:\Users\HP\Desktop\AI\docs\AI ASS-11.2.py"
Key found
Key deleted
Key not found
Key not found
PS C:\Users\HP\Desktop\AI>
```

